Practical : 3	**Implement A* or heuristic search for path planning and evaluate the effect of different heuristics.**
Date : 10/08/26	

Aim: -

To implement the A* heuristic search algorithm for path planning and evaluate the effect of different heuristics using a Hospital Delivery Robot example.

Algorithm:-

1. Start the program.
2. Create a graph representing different locations in the hospital.
3. Define the travelling cost between connected locations.
4. Assign coordinates to each hospital location.
5. Select the start location and goal location.
6. Choose a heuristic function: Zero Heuristic, Manhattan Distance, or Euclidean Distance.
7. Calculate the heuristic value h(n).
8. Calculate the actual path cost g(n).
9. Calculate the total estimated cost using:

f(n) = g(n) + h(n)

10. Add the starting node to the priority queue.
11. Select the node with the lowest f(n) value.
12. Check whether the selected node is the goal.
13. If it is the goal, return the path and total cost.
14. Otherwise, explore all neighbouring nodes.
15. Update the cost if a better path is found.
16. Repeat until the goal is reached or no path exists.
17. Compare the results of Zero, Manhattan, and Euclidean heuristics.
18. Display the path, cost, nodes explored, and execution time.
19. Stop the program.

Program:-

```
from flask import Flask, render_template, request
import heapq
import math
import time
import datetime

app = Flask(__name__)

# Hospital Map
hospital_graph = {
    "Pharmacy": {
        "Reception": 2,
        "Ward_A": 4
    },
    "Reception": {
        "Pharmacy": 2,
        "Ward_B": 3,
        "Emergency": 5
    },
    "Ward_A": {
        "Pharmacy": 4,
        "Ward_B": 2,
        "ICU": 7
    },
    "Ward_B": {
        "Reception": 3,
        "Ward_A": 2,
        "Ward_C": 3
    },
    "Emergency": {
        "Reception": 5,
        "Ward_C": 2
    },
    "Ward_C": {
        "Ward_B": 3,
        "Emergency": 2,
        "ICU": 3
    },
    "ICU": {
```

```
        "Ward_A": 7,
        "Ward_C": 3
    }
}

# Coordinates of Hospital Locations
coordinates = {
    "Pharmacy": (0, 0),
    "Reception": (1, 1),
    "Ward_A": (1, -1),
    "Ward_B": (2, 0),
    "Emergency": (2, 2),
    "Ward_C": (3, 1),
    "ICU": (4, 0)
}


# Manhattan Distance
def manhattan_distance(current, goal):
    x1, y1 = coordinates[current]
    x2, y2 = coordinates[goal]

    return abs(x1 - x2) + abs(y1 - y2)


# Euclidean Distance
def euclidean_distance(current, goal):
    x1, y1 = coordinates[current]
    x2, y2 = coordinates[goal]

    return math.sqrt(
        (x1 - x2) ** 2 +
        (y1 - y2) ** 2
    )


# Zero Heuristic
def zero_heuristic(current, goal):
    return 0


# Select Heuristic
```

```
def calculate_heuristic(current, goal, heuristic):

    if heuristic == "manhattan":
        return manhattan_distance(current, goal)

    elif heuristic == "euclidean":
        return euclidean_distance(current, goal)

    else:
        return zero_heuristic(current, goal)


# A* Search Function
def a_star(start, goal, heuristic):

    start_time = time.perf_counter()

    priority_queue = []

    g_cost = 0

    h_cost = calculate_heuristic(
        start,
        goal,
        heuristic
    )

    f_cost = g_cost + h_cost

    heapq.heappush(
        priority_queue,
        (
            f_cost,
            g_cost,
            start,
            [start]
        )
    )

    best_cost = {
        start: 0
    }
```

```
explored = set()

explored_count = 0

while priority_queue:

    current_f, current_g, current_node, path = \
        heapq.heappop(priority_queue)

    if current_node in explored:
        continue

    explored.add(current_node)

    explored_count += 1

    # Check Goal
    if current_node == goal:

        execution_time = (
            time.perf_counter() - start_time
        )

        return {
            "path": path,
            "cost": current_g,
            "nodes": explored_count,
            "time": execution_time
        }

    # Explore Neighbours
    for neighbour, edge_cost in \
            hospital_graph[current_node].items():

        new_g = current_g + edge_cost

        if neighbour not in best_cost or \
                new_g < best_cost[neighbour]:

            best_cost[neighbour] = new_g
```

```
            h = calculate_heuristic(
                neighbour,
                goal,
                heuristic
            )

            f = new_g + h

            heapq.heappush(
                priority_queue,
                (
                    f,
                    new_g,
                    neighbour,
                    path + [neighbour]
                )
            )

    execution_time = (
        time.perf_counter() - start_time
    )

    return {
        "path": [],
        "cost": None,
        "nodes": explored_count,
        "time": execution_time
    }


# Compare Different Heuristics
def compare_heuristics(start, goal):

    results = []

    heuristics = [
        ("zero", "Zero Heuristic"),
        ("manhattan", "Manhattan Distance"),
        ("euclidean", "Euclidean Distance")
    ]

    for key, name in heuristics:
```

```
        result = a_star(
            start,
            goal,
            key
        )

        results.append({
            "name": name,
            "path": result["path"],
            "cost": result["cost"],
            "nodes": result["nodes"],
            "time": result["time"]
        })

    return results


# Home Page
@app.route("/", methods=["GET", "POST"])
def index():

    locations = list(
        hospital_graph.keys()
    )

    result = None
    comparison = None

    current_date = datetime.datetime.now()

    if request.method == "POST":

        start = request.form["start"]

        goal = request.form["goal"]

        heuristic = request.form["heuristic"]

        # Run A*
        result = a_star(
            start,
```

```
            goal,
            heuristic
        )

        # Compare all heuristics
        comparison = compare_heuristics(
            start,
            goal
        )

    return render_template(
        "index.html",
        locations=locations,
        result=result,
        comparison=comparison,
        date=current_date.strftime(
            "%d-%m-%Y %H:%M:%S"
        )
    )


# Run Flask Application
if __name__ == "__main__":

    print("=" * 60)
    print("A* Hospital Delivery Robot Started")
    print("=" * 60)
    print("Open browser:")
    print("http://127.0.0.1:5000")

    app.run(debug=True)
```

templates/index.html

```
<!DOCTYPE html>
<html>
<head>

  <title>A* Hospital Delivery Robot</title>

  <style>

    body {
      font-family: Arial, sans-serif;
      background-color: #f2f2f2;
      margin: 0;
      padding: 30px;
    }

    .container {
      max-width: 900px;
      margin: auto;
      background: white;
      padding: 30px;
      border-radius: 10px;
    }

    h1 {
      text-align: center;
    }

    .description {
      text-align: center;
      margin-bottom: 30px;
    }

    label {
      font-weight: bold;
    }

    select {
      width: 100%;
      padding: 10px;
```

```
        margin-top: 8px;
        margin-bottom: 20px;
    }

    button {
        width: 100%;
        padding: 12px;
        background-color: #333;
        color: white;
        border: none;
        cursor: pointer;
        font-size: 16px;
    }

    .result {
        margin-top: 30px;
        padding: 20px;
        background-color: #eeeeee;
        border-radius: 8px;
    }

    table {
        width: 100%;
        border-collapse: collapse;
        margin-top: 20px;
    }

    th, td {
        border: 1px solid #999;
        padding: 10px;
        text-align: center;
    }

    th {
        background-color: #dddddd;
    }

  </style>

</head>

<body>
```

```
<div class="container">

  <h1>A* Hospital Delivery Robot</h1>

  <p class="description">
    Find the best path from the Pharmacy
    to the destination using A* Search.
  </p>

  <form method="POST">

    <label>Start Location</label>

    <select name="start">
      {% for location in locations %}
        <option value="{{ location }}">
          {{ location }}
        </option>
      {% endfor %}
    </select>


    <label>Goal Location</label>

    <select name="goal">
      {% for location in locations %}
        <option value="{{ location }}">
          {{ location }}
        </option>
      {% endfor %}
    </select>


    <label>Select Heuristic</label>

    <select name="heuristic">
      <option value="zero">Zero Heuristic</option>
      <option value="manhattan">Manhattan Distance</option>
      <option value="euclidean">Euclidean Distance</option>
    </select>
```

```
    <button type="submit">
        Find Best Path
    </button>

</form>


{% if result %}

<div class="result">

    <h2>A* Result</h2>

    <p>
        <strong>Path:</strong>
        {{ result.path | join(" → ") }}
    </p>

    <p>
        <strong>Total Path Cost:</strong>
        {{ result.cost }}
    </p>

    <p>
        <strong>Nodes Explored:</strong>
        {{ result.nodes }}
    </p>

    <p>
        <strong>Execution Time:</strong>
        {{ "%.6f"|format(result.time) }} seconds
    </p>

</div>

{% endif %}


{% if comparison %}

<div class="result">
```

```
    <h2>Heuristic Comparison</h2>

    <table>

      <tr>
        <th>Heuristic</th>
        <th>Path</th>
        <th>Cost</th>
        <th>Nodes Explored</th>
        <th>Time</th>
      </tr>

      {% for item in comparison %}

      <tr>
        <td>{{ item.name }}</td>
        <td>{{ item.path | join(" → ") }}</td>
        <td>{{ item.cost }}</td>
        <td>{{ item.nodes }}</td>
        <td>{{ "%.6f"|format(item.time) }}</td>
      </tr>

      {% endfor %}

    </table>

  </div>

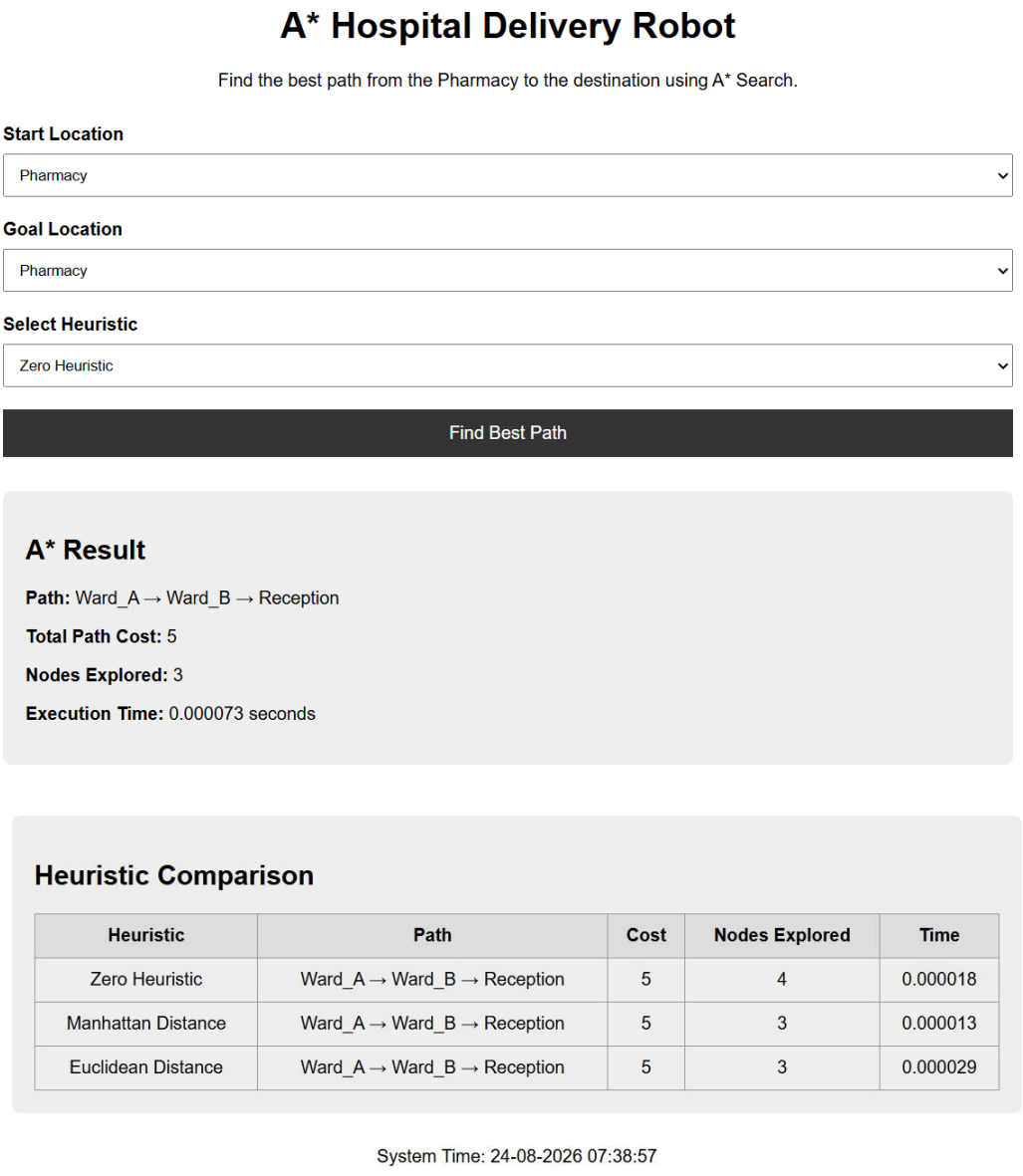
  {% endif %}

  <p style="text-align:center; margin-top:30px;">
    System Time: {{ date }}
  </p>

</div>

</body>
</html>
```

Output:-



Conclusion:-

Thus, the A* heuristic search algorithm was successfully implemented in Python for a Hospital Delivery Robot to find the optimal path from the Pharmacy to the ICU. Different heuristics, namely Zero Heuristic, Manhattan Distance, and Euclidean Distance, were compared based on path cost, number of explored nodes, and execution time. The practical demonstrates how heuristics can guide the search process and improve path planning efficiency.